

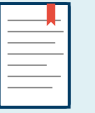
VS Code Copilot Customization Primitives

A reference guide to the building blocks for customizing GitHub Copilot in VS Code: **instructions, prompts, skills, agents (custom modes), hooks, tools, context, and memory.**

Visual Overview



Comparison Table

	.instructions.md	.prompt.md	SKILL.md	.agent.md (modes)	Hooks	Tools	Context	Memory
What it is	Standing rules / conventions	Saved, reusable chat command	Agent-routed workflow + domain knowledge	Custom chat mode (persona + tools + model)	Event-driven triggers around agent actions	Callable capabilities (built-in, MCP, extension)	Files / symbols / selections attached to a turn	Persistent notes across turns and sessions
Analogy — the Copilot Agent is the Chef 🍳	 House rules pinned in the kitchen. The chef silently obeys them whenever relevant.	 Standing order ticket ("the usual"). Hand it in — the chef cooks the same dish the same way.	 Recipe card in the cookbook. Pulled off the shelf only when a customer orders that dish.	 Uniform / station. Sushi chef vs. pastry chef — different specialty, different tools allowed.	 Smoke alarm / timer. Fires automatically when something happens — alerts, dings, or blocks.	 Kitchen appliances — oven, knife, mixer. Without them the chef can plan but can't cook.	 Ingredients on the counter. The chef can only cook with what's in reach right now.	 Personal notebook of regulars' preferences and lessons ("oven runs 10°F hot").
Function (plain English)	The "house rules." A short list of dos and don'ts that Copilot silently follows whenever it touches matching files. You write the rule once; it's enforced on every turn that involves those files, without you mentioning it again. Think of it as a coding-style poster pinned to the wall.	A "saved prompt template" you can fire like a macro. Instead of typing the same long instruction every time, you save it once and trigger it with <code>/name</code> . It can take parameters and runs as if you typed it.	A "playbook" for the agent. A complete how-to for one task. The agent reads only the short description up front, and pulls in the full playbook only when your request matches. Lets you pack lots of know-how without bloating every prompt.	A "named persona" with its own job description, allowed tools, and chosen model. Switching modes is like swapping out the whole assistant — e.g., a strict read-only "Code Reviewer" mode vs. a free-roaming "Refactor" mode.	A "tripwire" that runs your script when something happens in the agent's lifecycle. Used to enforce policy, log activity, run formatters, or block unsafe actions automatically.	The actual "verbs" the agent can perform — read a file, run a command, search the web, query a DB. Without tools, the agent can only talk; with tools, it can act.	The "evidence on the table" for one turn — the files, selections, terminal output, screenshots, etc. that the agent is allowed to look at right now.	The agent's "notebook." Lets it remember things between turns and sessions: your preferences, project conventions, an in-progress plan. Without memory, every conversation starts from zero.
Examples	<code>typescript.instructions.md (applyTo: "**/*.ts")</code> — "Prefer <code>interface</code> over <code>type</code>; never use <code>any</code>." <code>tests.instructions.md (applyTo: "**/*.test.*")</code> — "Use	<code>/release-notes</code> — generates a CHANGELOG entry from the staged diff. <code>/explain-selection</code> — explains the selected code line-by-line. <code>/scaffold-component</code> — asks for a name, creates component +	<code>add-migration</code> — full Prisma/Knex migration procedure (file, schema, up/down, tests). <code>review-pr</code> — checkout branch, diff main, run lint/tests, post comment. <code>bump-</code>	<code>security-reviewer.agent.md</code> — read-only tools, GPT-4.1, OWASP focus. <code>architect.agent.md</code> — diagram + research only; produces design docs. <code>tutor.agent.md</code> — explains	- Pre-tool hook on <code>run_in_terminal</code> — block <code>rm -rf /</code> or <code>git push --force</code>. - Post-edit hook — auto-run <code>prettier</code> + <code>eslint</code> --fix. - On-save hook — scan for secrets, reject save if	- Built-in: <code>read_file</code>, <code>grep_search</code>, <code>run_in_terminal</code>. - MCP: GitHub server — <code>create_issue</code>, <code>list_pull_requests</code>, <code>merge_pr</code>. - MCP: Postgres server — <code>run_query</code>,	- The file currently open in the editor (auto). - A code selection you highlighted before asking (auto). <code>#file:src/app/users.ts</code> referenced in the chat box. - Output of the last terminal	<code>/memories/preferences.md</code> — "Use 2-space indent. Prefer <code>pnpm</code>. TS strict." <code>/memories/debugging.md</code> — "ESM in Jest? Check <code>transformIgnorePatterns</code>." <code>/memories/re</code>

	.instructions.md	.prompt.md	SKILL.md	.agent.md (modes)	Hooks	Tools	Context	Memory
	Vitest. Arrange/Act/Assert. No snapshot tests." • <code>security.instructions.md</code> (<code>applyTo: ""</code>) — "Always parameterize SQL. Never log secrets." • <code>react.instructions.md</code> (<code>applyTo: "**/*.tsx"</code>) — "Functional components only." • <code>commit-msgs.instructions.md</code> — "Use Conventional Commits."	test + story. • <code>/triage-issue</code> — fetches a GitHub issue, summarises, suggests labels. • <code>/sql-from-nl</code> — takes <code>\${input:question}</code>, returns a SQL query.	dependency — update lockfile, rebuild, run tests, open PR. • <code>incident-postmortem</code> — pull logs, build timeline, draft postmortem. • <code>localize-strings</code> — extract hard-coded strings to i18n bundle.	concepts, never edits, asks Socratic questions. • <code>release-manager.agent.md</code> — has <code>git</code>, <code>gh</code>, CI tools. • <code>data-analyst.agent.md</code> — DB + notebook tools; restricted from app code.	found. • Pre-commit hook — verify changed files have tests + docs. • Audit hook — append every tool call to <code>.copilot/audit.log</code>.	<code>describe_table</code>. • Extension-contributed: Jira extension adding <code>create_jira_ticket</code>. • Browser tools: <code>open_browser_page</code>, <code>click_element</code>.	command via <code>#terminalLastCommand</code>. • A screenshot dragged into chat ("why does this UI look broken?"). • <code>@workspace</code> to search the whole repo.	<code>po/conventions.md</code> — "Components in <code>src/ui/</code>, state in <code>src/store/</code>." • <code>/memories/repo/build.md</code> — "Build: <code>pnpm build</code>. Test: <code>pnpm test --run</code>. Node 20." • <code>/memories/session/plan.md</code> — current task checklist, kept up to date.
How invoked	Auto-attached when <code>applyTo</code> glob matches	User runs it: <code>/name</code> , palette, editor play, <code># -ref</code>	Agent reads <code>description</code> and loads body via <code>read_file</code>	User picks the mode from the chat mode dropdown	Fired automatically by lifecycle events	Called by the agent (or by a skill/prompt)	Auto-attached or user-attached (<code>#file</code> , drag-drop)	Agent reads/writes via memory tool; user-tier head auto-loaded
When invoked	Every chat turn whose context matches the glob	On demand by the user	When user's request semantically matches the description	For the entire duration the mode is selected	At specific lifecycle points (before/after tool, on save, etc.)	Per agent turn, as needed to complete the task	Every turn (whatever is currently attached)	Auto snippet every turn; full files on explicit read
How used	Prepended to system prompt as standing guidance	Body becomes the user message (variables resolved)	Body loaded into context, then followed as a procedure	Replaces default system prompt + restricts tools	Run scripts / validations / notifications around activity	Perform read/write/search/execute actions	Provides the <i>content</i> the model reasons over	Carries facts / preferences / plans forward
Scope	Workspace or user; narrowed by <code>applyTo</code>	Workspace or user; available in any chat session	Workspace or user; globally available to the agent	Workspace or user; active only while selected	Workspace or user; scoped by event filter	Session-wide; gateable per mode/skill	Per turn	3 tiers: user (global), session (this convo), repo (this workspace)
Loaded	Every matching turn (cheap; keep short)	Only when user invokes	Frontmatter always; body lazy-loaded on agent decision	Loaded when mode is active	Loaded by runtime; executes only on event	Schema known; body runs on call	Only what's attached	User-tier: first 200 lines auto; others on demand
Typical content	"Prefer <code>const</code> ; never <code>any</code> ; use <code>tabs</code> "	"Generate release notes from <code>\${input:range}</code> "	Multi-step procedure: "Add a DB migration end-to-end"	"You are a security reviewer. Tools: read-	Lint after edit, block secret commits, log tool calls	<code>read_file</code> , <code>grep_search</code> , <code>run_in_term</code>	Source files, diffs, terminal output, web pages, images	Coding preferences, project conventions,

	.instructions.md	.prompt.md	SKILL.md	.agent.md (modes)	Hooks	Tools	Context	Memory
				only. Model: GPT-4.1."		inal , MCP servers		in-progress plan
Frontmatter	applyTo , description	mode , model , tools , description	name , description (routing-critical), tools	name , description , tools , model	event , matches , command	Declared by extension/MCP manifest	—	—
Storage in repo	.github/instructions/*.instructions.md	.github/prompts/*.prompt.md	.github/skills/<name>/SKILL.md + supporting files	.github/chatmodes/*.agent.md	.github/hooks/* or settings entries	MCP via .vscode/mcp.json ; extensions via package.json	Not stored — derived live	/memories/repo/ is workspace-local
Storage user-global	User/prompts/*.instructions.md	User/prompts/*.prompt.md	User/prompts/skills/<name>/SKILL.md	User/prompts/*.agent.md	User settings.json or hooks folder	User settings.json / extensions / user mcp.json	—	/memories/ + /memories/session/
Discovery setting	chat.instructionsFileLocations	chat.promptFileLocations , chat.promptFiles	Skill folders auto-discovered	Mode files auto-discovered	Hook config in settings	chat.mcp.enabled , extension activation	Built-in	Built-in (memory tool)
Who decides	VS Code (glob match)	User	Agent (model)	User	Runtime (event)	Agent	User + VS Code	Agent (writes); auto + agent (reads)

Quick Mental Model

Picture a busy restaurant kitchen. The **Copilot Agent** is the 🍳 **chef**. Everything else is the stuff around the chef:

- **Instructions** — standing rules (auto, passive). 📋 *House rules pinned to the kitchen wall.*
- **Prompts** — saved commands you fire (user-driven). 📄 *A standing order ticket — "the usual."*
- **Skills** — capabilities the agent picks up when relevant (agent-driven). 📖 *A recipe card pulled off the shelf when the right dish is ordered.*
- **Agents / modes** — whole personas with their own toolset and system prompt (user-selected). 👤 *A different chef in a different uniform — sushi vs. pastry.*
- **Hooks** — automation around the agent's lifecycle (event-driven). 🔔 *The smoke alarm or kitchen timer that fires by itself.*
- **Tools** — the verbs the agent can actually perform. 🔪 *The knives, oven, mixer — without them the chef can plan but not cook.*
- **Context** — the nouns the agent reasons over for one turn. 🍎 *The ingredients on the counter right now.*
- **Memory** — the long-term notebook that survives turns and sessions. 📓 *The chef's personal notebook of regulars' preferences and lessons learned.*